

VEXTA NETWORK

ENTREGA TÉCNICA Y GUÍA DE ADMINISTRACIÓN

TIPO DE DOCUMENTO

Guía de Entrega Técnica

FECHA DE ENTREGA

Julio 2026

SERVICIOS TOTALES

**Prisma / MongoDB / Plisio /
Resend**

ESTADO DE IMPLEMENTACIÓN

✓ Producción Activa

Recursos Completos de Auditoría

Vexta Network — Guía de Entrega Técnica y Administración del Sistema

Este documento contiene los detalles de la arquitectura del sistema, guías de configuración de la base de datos, arquitecturas del servidor e instrucciones operativas para la plataforma Vexta Network. Está diseñado para ayudar a otro desarrollador a auditar y asumir la gestión del proyecto.

1. Descripción General del Proyecto

Vexta es una plataforma de Arbitraje de IA y Trading construida con una pila moderna de TypeScript/React.

- **Framework Principal:** Next.js 16 (React 19, TailwindCSS, TypeScript)
 - **Capa de Base de Datos:** Prisma ORM (v6.19.0) con adaptador de MongoDB
 - **Servicio de Correo Electrónico:** Resend (correos electrónicos transaccionales y OTPs)
 - **Pasarela de Pago Cripto:** Plisio (USDT BEP-20 en Binance Smart Chain)
 - **Dominio de Producción:** <https://vexta.network>
-

2. Configuración de la Base de Datos

La plataforma utiliza **Prisma ORM** conectándose a un **Cluster de MongoDB Atlas**.

Plantilla de Cadena de Conexión

Configure la variable de entorno `DATABASE_URL` dentro de su archivo `.env` local utilizando la siguiente estructura:

```
DATABASE_URL="mongodb+srv://[username]:[password]@[host]/vexta?retryWrites=true"
```

- **Adaptador / Esquema:** `prisma/schema.prisma` define las colecciones:
 - `User` : Cuentas principales, seguimiento de saldos, referencias de línea descendente (downline), parámetros MLM.
 - `Plan` : Niveles de inversión Starter, Advance y Ultra.
 - `Investment` : Contratos activos, capital, días transcurridos y registros de ROI.
 - `DailyROIEntry` : Filas de auditoría para distribuciones diarias de ganancias.
 - `PlisioInvoice` : Facturas de transacciones generadas para depósitos de usuarios.
 - `BatchPayoutRun` : Registro de generaciones de archivos CSV de pago por lotes de los viernes.
 - `ReferralLink` / `Commission` : Estructura MLM y pagos de niveles.
 - `Transaction` / `Withdrawal` : Registros financieros y estados de solicitud de retiro.
 - `Settings` : Conmutadores globales del sistema (modo de mantenimiento, control de pasarela, reglas de promoción).

3. Infraestructura del Servidor y SSL

- **Proveedor de Hosting:** El frontend y las rutas de API están alojados actualmente en **Vercel** (configuraciones de `vercel.json`). La pila también se puede implementar en un VPS Ubuntu estándar con Node.js 18+ (requiere configuraciones estándar para PM2 o Systemd).
 - **Certificados SSL:** Los certificados SSL se aprovisionan de forma dinámica a través de la plataforma de hosting (Vercel / Cloudflare). No se requieren instalaciones manuales de certificados.
 - **Resolución DNS:** Administrado a través del registrador de dominios apuntando a los servidores de nombres de Vercel (`ns1.vercel-dns.com` / `ns2.vercel-dns.com`).
-

4. Integraciones de Terceros

1. Resend (Integración de Correo Electrónico)

- **Clave de API:** [Proporcionada de forma segura por el propietario]
- **Verificación de Dominio:** El dominio del remitente (ej. `vexta.network`) debe verificarse mediante registros SPF/DKIM dentro del panel de control de Resend.
- **Características:** Envía códigos de verificación de cuenta (OTP), confirmaciones de retiro, actualizaciones de soporte y registros de informes de ejecución de tareas cron diarias.

2. Plisio (Pasarela de Pago USDT BEP-20)

- **Clave Secreta:** [Proporcionada de forma segura por el propietario]
- **Auto-Envío (Auto-Forwarding):** Configure su billetera corporativa de destino en la configuración del panel de control de Plisio. Plisio redirige automáticamente los pagos entrantes de los clientes a esta dirección.
- **URL de Webhook:** `https://[su-dominio]/api/plisio/webhook`
- **Mecanismo Robustecido de Verificación:**
 - La ruta del webhook `app/api/plisio/webhook/route.ts` admite validaciones estándar de firma IPN.
 - Para maximizar la seguridad y evitar fallos por variaciones en la estructura de los datos (payload), el sistema realiza una **llamada de contingencia directa a la API del backend** a `https://api.plisio.net/api/v1/invoices/${txn_id}?api_key=${secretKey}` para consultar directamente a los servidores de Plisio el estado final y oficial de la transacción.

5. Lógica de Negocio Central y Algoritmos

Un desarrollador que se haga cargo de la plataforma debe comprender los siguientes tres sistemas:

1. Distribución Diaria de ROI e Interés Compuesto

- **Ruta:** `/api/admin/run-daily-roi` (activada de lunes a viernes).

- **Autorización:** Restringida por sesión de administrador JWT O pasando la clave `CRON_SECRET` a través de la cabecera `x-cron-key` o el parámetro de consulta `?secret=`.
- **Lógica** (`server/services/earnings.service.ts`):
 - Calcula un `1.0%` de interés diario.
 - Incorpora interés compuesto con un **retraso pendiente de 2 días**: los retornos distribuidos hoy se agregan al `pendingCapital` (capital pendiente) y solo se fusionan en la base de interés compuesto (`activeCapital`) después de 2 días hábiles.
 - Cada inversión tiene un **límite estricto** de pago del `200%` (principal × 2). Una vez que la suma de todas las ganancias (ROI + comisiones MLM) alcanza este tope, el estado del contrato pasa a `completed` (completado) y se detiene la generación de ganancias.

2. Propagación de Comisiones Unilevel del Sistema MLM

- **Activación:** Se ejecuta automáticamente tras la activación exitosa de un paquete de inversión de usuario.
- **Lógica** (`lib/referral-engine.ts`):
 - Distribuye comisiones a lo largo de la cadena de patrocinadores para **13 niveles** ascendentes:
 - Nivel 1: 8% | Nivel 2: 5% | Nivel 3: 3% | Nivel 4: 3% | Nivel 5: 2% | Nivel 6: 2% | Nivel 7: 1% | Niveles 8–13: 0.5%.
 - Enforma el **Tope Estricto del 200%**: Cuando una comisión se propaga a un patrocinador, se evalúa la capacidad de los paquetes activos de dicho patrocinador. Las comisiones que caben dentro de la capacidad disponible se agregan a su saldo; cualquier excedente se desborda y se **pierde permanentemente**.
 - Registra eventos de auditoría independientes para comisiones brutas generadas y comisiones netas acreditadas reales.

3. Procesamiento Administrativo de Pagos por Lotes

- **Ruta:** `app/api/admin/batch-payout/route.ts`
- **Flujo de trabajo:**
 1. El administrador solicita la generación de un lote (`action: "generate"`).
 2. El sistema escanea todas las solicitudes de retiro pendientes y calcula el Pago Neto tras restar las tarifas de procesamiento (6% para retiros inferiores a \$600, 2% para \$600 o más).

3. Formatea y exporta un archivo CSV que coincide con el protocolo de pago masivo de Plisio: `address,amount,currency`.
4. La ejecución del pago (`action: "execute"`) está protegida por un código OTP de dos factores (2FA) enviado al correo electrónico del administrador. Al ejecutarse, los estados de retiro se marcan como `approved` (aprobados) y se registran los asientos contables correspondientes.

6. Scripts de Utilidad Administrativos y de Base de Datos

El directorio `scripts/` contiene scripts de ayuda estándar para la gestión de la base de datos y operaciones de auditoría:

- `scripts/check-plisio-api.ts`: Prueba la salud del endpoint de Plisio y devuelve los estados de las facturas.
- `scripts/check-user-deposit.ts`: Comprueba manualmente el estado de una ID de transacción en la blockchain.
- `scripts/clean-db.ts`: Restablece todos los saldos de los usuarios a 0 (útil para restablecimientos de pruebas de entorno).
- `scripts/clean-db-delete.ts`: Elimina entradas de prueba.
- `scripts/credit-mismatch-deposit.ts`: Corrige depósitos que generaron discrepancias en los importes de Plisio.
- `scripts/reset-data.ts`: Purga completamente los usuarios e inversiones para restablecer la base de datos.
- `scripts/seed-plans.ts`: Registra o actualiza los contratos predeterminados STARTER, ADVANCE y ULTRA.

Para ejecutar cualquier script (usando ts-node):

```
npx ts-node --project tsconfig.json scripts/check-plisio-api.ts
```

7. Guía de Desarrollo Local e Implementación

Para que un nuevo desarrollador configure la base de código localmente:

Instalación de Prerrequisitos

Asegúrese de tener Node.js 18+ y npm/pnpm/yarn están instalados en el sistema local.

```
# 1. Instalar dependencias
npm install

# 2. Configurar Variables de Entorno Locales
# Copie la plantilla del entorno y complete los secretos (proporcionados de t
cp .env.example .env

# 3. Generar el Cliente Prisma
# Construye localmente los tipos del cliente Prisma basados en schema.prisma
npx prisma generate

# 4. Sembrar Base de Datos (Seed)
# Siembra los niveles de inversión predeterminados y configura la cuenta de a
# Correo del Admin: admin@vexta.app | Contraseña: Admin@1234!
npx prisma db seed

# 5. Iniciar Servidor de Desarrollo Local
# Inicia la aplicación en http://localhost:3000
npm run dev
```

Implementación en Producción

Para compilar el paquete next estático y ejecutarlo en producción:

```
# Compilación de Producción
npm run build

# Iniciar Servidor de Producción
npm start
```

Nota: Asegúrese de que todas las variables de entorno (claves del archivo .env) estén configuradas en el panel de control de su plataforma de hosting.